

Sogang University

Dynamic Memory Allocation: Advanced Concepts

CSE4100: System Programming

Youngjae Kim (PhD)
<https://sites.google.com/site/youkim/home>
 Data-Intensive AI Computing and Systems Laboratory (DISCOS)
<https://discos.sogang.ac.kr>
 Office: R911, E-mail: youkim@sogang.ac.kr

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

1

1

Sogang University

Today

- Explicit free lists
- Segregated free lists
- Garbage collection
- Memory-related perils and pitfalls

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

2

2

Sogang University

Keeping Track of Free Blocks

- Method 1: **Implicit free list** using length—links all blocks



- Method 2: **Explicit free list** among the free blocks using pointers



- Method 3: **Segregated free list**
 - Different free lists for different size classes
- Method 4: **Blocks sorted by size**
 - Can use a balanced tree (e.g. Red-Black tree) with pointers within each free block, and the length used as a key

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

3

3

Sogang University

Explicit Free Lists

Allocated (as before)

Size	a
Payload and padding	
Size	a

Free

Size	a
Next	
Prev	
Size	a

- Maintain list(s) of **free** blocks, not **all** blocks
 - The "next" free block could be anywhere
 - So we need to store forward/back pointers, not just sizes
 - Still need boundary tags for coalescing
 - Luckily we track only free blocks, so we can use payload area

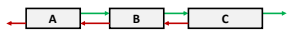
Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

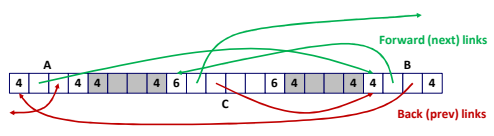
4

4

Sogang University

Explicit Free Lists

- Logically:
 
- Physically: blocks can be in any order



Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

5

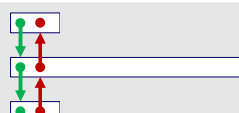
5

Sogang University

Allocating From Explicit Free Lists

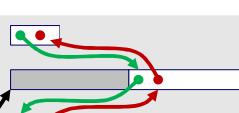
conceptual graphic

Before



After

(with splitting)



= malloc (...)

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

6

6

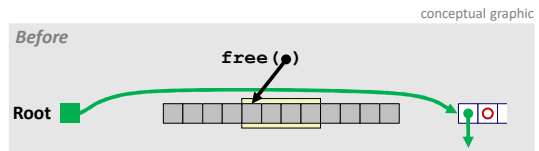
Freeing With Explicit Free Lists

- **Insertion policy:** Where in the free list do you put a newly freed block?
- **LIFO (last-in-first-out) policy**
 - Insert freed block at the beginning of the free list
 - **Pro:** simple and constant time
 - **Con:** studies suggest fragmentation is worse than address ordered
- **Address-ordered policy**
 - Insert freed blocks so that free list blocks are always in address order:

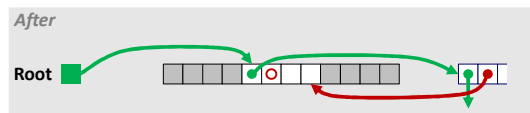
$$\text{addr}(\text{prev}) < \text{addr}(\text{curr}) < \text{addr}(\text{next})$$
 - **Con:** requires search
 - **Pro:** studies suggest fragmentation is lower than LIFO

7

Freeing With a LIFO Policy (Case 1)

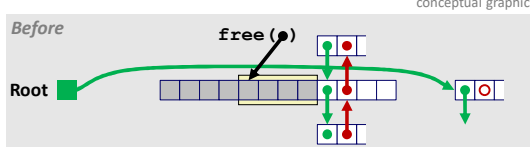


- Insert the freed block at the root of the list

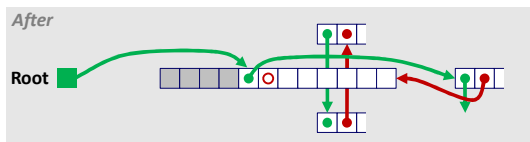


8

Freeing With a LIFO Policy (Case 2)

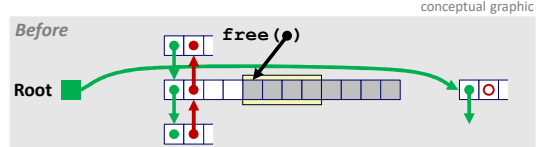


- Splice out successor block, coalesce both memory blocks and insert the new block at the root of the list

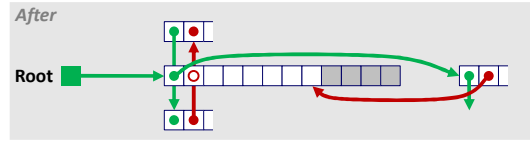


9

Freeing With a LIFO Policy (Case 3)

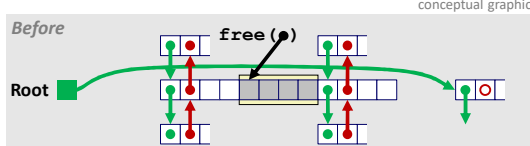


- Splice out predecessor block, coalesce both memory blocks, and insert the new block at the root of the list

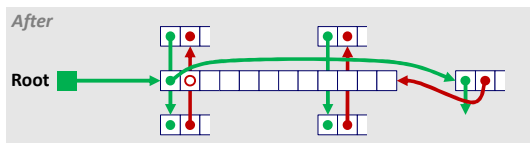


10

Freeing With a LIFO Policy (Case 4)



- Splice out predecessor and successor blocks, coalesce all 3 memory blocks and insert the new block at the root of the list



11

Explicit List Summary

- **Comparison to implicit list:**
 - Allocate is linear time in number of **free** blocks instead of **all** blocks
 - **Much faster** when most of the memory is full
 - Slightly more complicated allocate and free since needs to splice blocks in and out of the list
 - Some extra space for the links (2 extra words needed for each block)
 - Does this increase internal fragmentation?
- **Most common use of linked lists is in conjunction with segregated free lists**
 - Keep multiple linked lists of different size classes, or possibly for different types of objects

12

Keeping Track of Free Blocks

- Method 1: **Implicit list** using length—links all blocks



- Method 2: **Explicit list** among the free blocks using pointers



- Method 3: **Segregated free list**

- Different free lists for different size classes

- Method 4: **Blocks sorted by size**

- Can use a balanced tree (e.g. Red-Black tree) with pointers within each free block, and the length used as a key

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

13

13

Today

- Explicit free lists
- Segregated free lists
- Garbage collection
- Memory-related perils and pitfalls

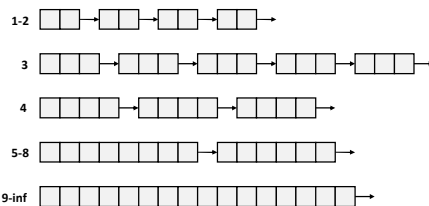
Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

14

14

Segregated List (Seglist) Allocators

- Each **size class** of blocks has its own free list



- Often have separate classes for each small size
- For larger sizes: One class for each two-power size

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

15

15

Seglist Allocator

- Given an array of free lists, each one for some size class

- To allocate a block of size n :

- Search appropriate free list for block of size $m > n$
- If an appropriate block is found:
 - Split block and place fragment on appropriate list (optional)
- If no block is found, try next larger class
- Repeat until block is found

- If no block is found:

- Request additional heap memory from OS (using `sbrk()`)
- Allocate block of n bytes from this new memory
- Place remainder as a single free block in largest size class.

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

16

16

Seglist Allocator (cont.)

- To free a block:

- Coalesce and place on appropriate list

- Advantages of seglist allocators

- Higher throughput
 - log time for power-of-two size classes
- Better memory utilization
 - First-fit search of segregated free list approximates a best-fit search of entire heap.
 - Extreme case: Giving each block its own size class is equivalent to best-fit.

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

17

17

More Info on Allocators

- D. Knuth, "The Art of Computer Programming", 2nd edition, Addison Wesley, 1973
 - The classic reference on dynamic storage allocation
- Wilson et al, "Dynamic Storage Allocation: A Survey and Critical Review", Proc. 1995 Int'l Workshop on Memory Management, Kinross, Scotland, Sept, 1995.
 - Comprehensive survey

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

18

18

Today

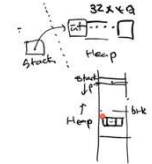
- Explicit free lists
- Segregated free lists
- Garbage collection
- Memory-related perils and pitfalls

19

Implicit Memory Management: Garbage Collection

- **Garbage collection:** automatic reclamation of heap-allocated storage—application never has to free

```
void foo() {
    int *p = malloc(128);
    return; /* p block is now garbage */
}
```



- Common in many dynamic languages:
 - Python, Ruby, Java, Perl, ML, Lisp, Mathematica
- Variants ("conservative" garbage collectors) exist for C and C++
 - However, cannot necessarily collect all garbage

20

Conservative Garbage Collector

- Conservative GC is a technique that mimics garbage collection in statically typed languages such as C and C++.

- Boehm-Demers-Weiser conservative GC (called "Boehm GC").

```
#include <gc.h>

int main() {
    GC_INIT(); // Initialize the collector
    int *p = (int *)GC_MALLOC(sizeof(int)); // Allocate memory
    *p = 42;
    // No need to free memory manually
}
```

- Pros:
 - Works with unmodified C/C++ code — no need to rewrite programs.
 - Helps avoid memory leaks in large or legacy codebases.
 - Can make C/C++ programming safer in some use cases.
- Cons:
 - Not 100% precise: may miss some garbage and cause "leaks".
 - Performance overhead compared to manual management.
 - Not suitable for real-time systems or highly memory-constrained environments.

21

Garbage Collection

- How does the memory manager know when memory can be freed?

- In general we cannot know what is going to be used in the future since it depends on conditionals
- But we can tell that certain blocks cannot be used if there are no pointers to them

- Must make certain assumptions about pointers

- Memory manager can distinguish pointers from non-pointers
- All pointers point to the start of a block
- Cannot hide pointers (e.g., by coercing them to an `int`, and then back again)

22

Classical GC Algorithms

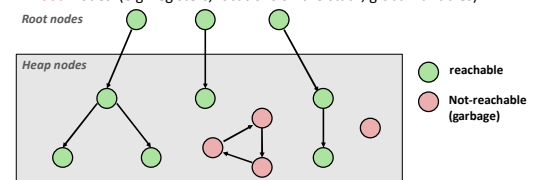
- Mark-and-sweep collection (McCarthy, 1960)
 - Does not move blocks (unless you also "compact")
- Reference counting (Collins, 1960)
 - Does not move blocks (not discussed)
- Copying collection (Minsky, 1963)
 - Moves blocks (not discussed)
- Generational Collectors (Lieberman and Hewitt, 1983)
 - Collection based on lifetimes
 - Most allocations become garbage very soon
 - So focus reclamation work on zones of memory recently allocated
- For more information: Jones and Lin, "Garbage Collection: Algorithms for Automatic Dynamic Memory", John Wiley & Sons, 1996.

23

Memory as a Graph

- We view memory as a directed graph

- Each block is a node in the graph
- Each pointer is an edge in the graph
- Locations not in the heap that contain pointers into the heap are called **root** nodes (e.g. registers, locations on the stack, global variables)



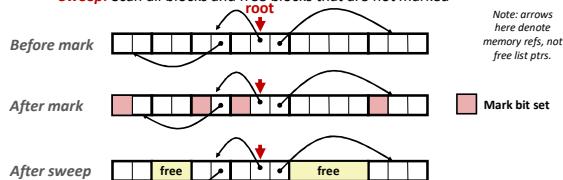
A node (block) is **reachable** if there is a path from any root to that node.

Non-reachable nodes are **garbage** (cannot be needed by the application)

24

Mark and Sweep Collecting

- Can build on top of malloc/free package
 - Allocate using malloc until you "run out of space"
- When out of space:
 - Use extra **mark bit** in the head of each block
 - Mark:** Start at roots and set mark bit on each reachable block
 - Sweep:** Scan all blocks and free blocks that are not marked



25

Assumptions For a Simple Implementation

- Application
 - new(n)**: returns pointer to new block with all locations cleared
 - read(b, i)**: read location *i* of block *b* into register
 - write(b, i, v)**: write *v* into location *i* of block *b*
- Each block will have a header word
 - addressed as *b[-1]*, for a block *b*
 - Used for different purposes in different collectors
- Instructions used by the Garbage Collector
 - is_ptr(p)**: determines whether *p* is a pointer
 - length(b)**: returns the length of block *b*, not including the header
 - get_roots()**: returns all the roots

26

Mark and Sweep (cont.)

p is a root pointer.

Mark using depth-first traversal of the memory graph

```
ptr mark(ptr p) {
    if (!is_ptr(p)) return; // do nothing if not pointer
    if (markBitSet(p)) return; // check if already marked
    setMarkBit(p); // set the mark bit
    for (i=0; i < length(p); i++) // call mark on all words
        mark(p[i]); // in the block
    return;
}
```

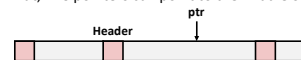
Sweep using lengths to find next block

```
ptr sweep(ptr p, ptr end) {
    while (p < end) {
        if (markBitSet(p))
            clearMarkBit(p);
        else if (allocateBitSet(p))
            free(p);
        p += length(p);
    }
}
```

27

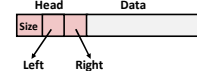
Conservative Mark & Sweep in C

- A "conservative garbage collector" for C programs
 - is_ptr()** determines if a word is a pointer by checking if it points to an allocated block of memory
 - But, in C pointers can point to the middle of a block



- So how to find the beginning of the block?

- Can use a balanced binary tree to keep track of all allocated blocks (key is start-of-block)
- Balanced-tree pointers can be stored in header (use two additional words)



Left: smaller addresses
Right: larger addresses

28

Today

- Explicit free lists
- Segregated free lists
- Garbage collection
- Memory-related perils and pitfalls

29

Memory-Related Perils and Pitfalls

- Dereferencing bad pointers
- Reading uninitialized memory
- Overwriting memory
- Referencing nonexistent variables
- Freeing blocks multiple times
- Referencing freed blocks
- Failing to free blocks

30

Sogang University

C operators

Operators
() [] -> .
! ~ ++ -- + - * & (type) sizeof
* / %
+ -
<< >>
< <= > >=
== !=
&
^
|
&&
||
?:
= += -= *= /= %= &= ^= != <<= >>=
,

Associativity
left to right
right to left
left to right
left to right
left to right
left to right
left to right
left to right
left to right
right to left
right to left
left to right

- >, (), and [] have high precedence, with * and & just below
- Unary +, -, and * have higher precedence than binary forms

Source: K&R page 53

31

Sogang University

C Pointer Declarations: Test Yourself!

int *p

p is a pointer to int

int *p[13]

p is an array[13] of pointer to int

int *(p[13])

p is an array[13] of pointer to int

int **p

p is a pointer to a pointer to an int

int (*p)[13]

p is a pointer to an array[13] of int

int *f()

f is a function returning a pointer to int

int (*f)()

f is a pointer to a function returning int

Source: K&R Sec 5.12

32

Sogang University

int *p

int *p[4]

int *(p[4])

Source: K&R Sec 5.12

33

Sogang University

int **p

int (*p)[4]

```
// int (*p)[13]
p -> [int, int, ..., int] // a single array of 13 integers

// int **p[13]
p[0] -> int // pointer to an int
p[1] -> int // pointer to an int
... total of 13 pointers to int
```

Source: K&R Sec 5.12

34

Sogang University

int *f()

int *p = f()

int (*f)()

f is a pointer to a function returning int

```
#include <stdio.h>

int say_hello() {
    printf("hello\n");
    return 42;
}

int main() {
    int (*f)(); // 함수 포인터 선언
    f = say_hello;

    int result = f(); // say_hello() 호출
    printf("result: %d\n", result); // 42 출력
    return 0;
}
```

Source: K&R Sec 5.12

35

Sogang University

Dereferencing Bad Pointers

- The classic scanf bug

```
int val;
...
scanf("%d", val);
```

Source: K&R Sec 5.12

36

6

Reading Uninitialized Memory

- Assuming that heap data is initialized to zero

```
/* return y = Ax */
int *matvec(int **A, int *x) {
    int *y = malloc(N*sizeof(int));
    int i, j;

    for (i=0; i<N; i++)
        for (j=0; j<N; j++)
            y[i] += A[i][j]*x[j];
    return y;
}
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

37

Reading Uninitialized Memory

- Assuming that heap data is initialized to zero

```
/* return y = Ax */
int *matvec(int **A, int *x) {
    int *y = malloc(N*sizeof(int));
    int i, j;

    for (i=0; i<N; i++)
        for (j=0; j<N; j++)
            y[i] += A[i][j]*x[j];
    return y;
}
```

- malloc() does not initialize the allocated memory.
- Use calloc() if initialization is required.

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

38

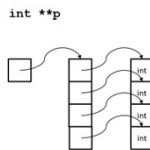
Overwriting Memory

- Allocating the (possibly) wrong sized object

```
int **p;

p = malloc(N*sizeof(int));

for (i=0; i<N; i++) {
    p[i] = malloc(M*sizeof(int));
}
```



Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

39

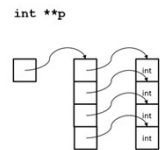
Overwriting Memory

- Allocating the (possibly) wrong sized object

```
int **p;

p = malloc(N*sizeof(int));

for (i=0; i<N; i++) {
    p[i] = malloc(M*sizeof(int));
}
```



- This is incorrect because p is of type int**, which means it should point to an array of int* (i.e., pointers to int).
- However, the code above allocates memory for N integers instead of N pointers to integers.

- Correct Version

```
p = malloc(N * sizeof(int *)); // Allocate memory for N pointers to int
for (int i = 0; i < N; i++) {
    p[i] = malloc(M * sizeof(int)); // Each pointer points to M ints
}
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

40

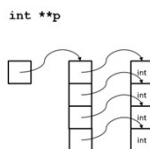
Overwriting Memory

- Off-by-one error

```
int **p;

p = malloc(N*sizeof(int *));

for (i=0; i<=N; i++) {
    p[i] = malloc(M*sizeof(int));
}
```



Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

41

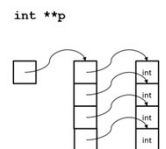
Overwriting Memory

- Off-by-one error

```
int **p;

p = malloc(N*sizeof(int *));

for (i=0; i<=N; i++) {
    p[i] = malloc(M*sizeof(int));
}
```



- Buffer overrun, leading to undefined behavior

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

42

Overwriting Memory

- Not checking the max string size

```
char s[8];
int i;

gets(s); /* reads "123456789" from stdin */
```

43

Overwriting Memory

- Not checking the max string size

```
char s[8];
int i;

gets(s); /* reads "123456789" from stdin */
```

- The gets() function does **not perform bounds checking**, so if the user inputs more than 7 characters (plus the null terminator), it will overflow the s[8] buffer, causing undefined behavior, such as:
 - ✓ Overwriting nearby variables (like i)
 - ✓ Crashing the program
 - ✓ Potential security vulnerabilities (e.g., buffer overflow attacks)

- Basis for classic buffer overflow attacks

44

Overwriting Memory

- Misunderstanding pointer arithmetic

```
int *search(int *p, int val) {
    while (*p && *p != val)
        p += sizeof(int);
    return p;
}
```

45

Overwriting Memory

- Misunderstanding pointer arithmetic

```
int *search(int *p, int val) {
    while (*p && *p != val)
        p += sizeof(int);
    return p;
}
```

- If sizeof(int) == 4, this line actually moves the pointer by 4 * sizeof(int) = 16 bytes, not 4.

- Correct Version

```
int *search(int *p, int val) {
    while (*p && *p != val)
        p++; // move to the next int
    return p;
}
```

46

Overwriting Memory

- Referencing a pointer instead of the object it points to

```
int *BinheapDelete(int **binheap, int *size) {
    int *packet;
    packet = binheap[0];
    binheap[0] = binheap[*size - 1];
    *size--;
    Heapify(binheap, *size, 0);
    return(packet);
}
```

47

Overwriting Memory

- Referencing a pointer instead of the object it points to

```
int *BinheapDelete(int **binheap, int *size) {
    int *packet;
    packet = binheap[0];
    binheap[0] = binheap[*size - 1];
    *size--;
    Heapify(binheap, *size, 0);
    return(packet);
}
```

- *(size--) // not (*size)--

- Correct Version

48

Referencing Nonexistent Variables

- Forgetting that local variables disappear when a function returns

```
int *foo () {
    int val;

    return &val;
}
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

49

49

Freeing Blocks Multiple Times

- Nasty!

```
x = malloc(N*sizeof(int));
    <manipulate x>
free(x);

y = malloc(M*sizeof(int));
    <manipulate y>
free(x);
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

50

50

Freeing Blocks Multiple Times

- Nasty!

```
x = malloc(N*sizeof(int));
    <manipulate x>
free(x);

y = malloc(M*sizeof(int));
    <manipulate y>
free(x);
```

- Tip (To avoid accidental double free)

```
free(x);
x = NULL;
```

- Calling free(x) twice:
 - ✓ program crash (segmentation fault)
 - ✓ heap corruption
 - ✓ security vulnerabilities.

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

51

51

Referencing Freed Blocks

- Evil!

```
x = malloc(N*sizeof(int));
    <manipulate x>
free(x);
...
y = malloc(M*sizeof(int));
for (i=0; i<M; i++)
    y[i] = x[i]++;
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

52

52

Failing to Free Blocks (Memory Leaks)

- Slow, long-term killer!

```
foo() {
    int *x = malloc(N*sizeof(int));
    ...
    return;
}
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

53

53

Failing to Free Blocks (Memory Leaks)

- Freeing only part of a data structure

```
struct list {
    int val;
    struct list *next;
};

foo() {
    struct list *head = malloc(sizeof(struct list));
    head->val = 0;
    head->next = NULL;
    <create and manipulate the rest of the list>
    ...
    free(head);
    return;
}
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

54

54

Dealing With Memory Bugs

- **Debugger: gdb**
 - Good for finding bad pointer dereferences
 - Hard to detect the other memory bugs
- **Data structure consistency checker**
 - Runs silently, prints message only on error
 - Use as a probe to zero in on error
- **Binary translator: valgrind**
 - Powerful debugging and analysis technique
 - Rewrites text section of executable object file
 - Checks each individual reference at runtime
 - Bad pointers, overwrites, refs outside of allocated block
- **glibc malloc contains checking code**
 - `setenv MALLOC_CHECK_ 3`
 - The malloc function in glibc (the GNU C Library) includes internal checking code to detect memory usage errors, such as double free, buffer overruns, and other memory corruption issues.
 - The MALLOC_CHECK_ environment variable controls how glibc's malloc behaves when it detects memory errors. The value determines the behavior: 0: Do nothing (default behavior).
 - ✓ 1: Print an error message to stderr
 - ✓ 2: Abort (terminate the program) immediately upon error
 - ✓ 3: Print an error message and then abort (combines 1 and 2)

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

55

55

valgrind

- **Memory Leak Detection**
 - Finds memory that was allocated (e.g., with malloc) but not properly freed before program termination
- **Invalid Memory Access Detection**
 - Detects use of
 - Uninitialized memory
 - Freed memory (use-after-free)
 - Out-of-bounds array access
 - Double free

```
int *p = malloc(10 * sizeof(int));
// forgot to free
```

```
valgrind --leak-check=full ./my_program
```

```
LEAK SUMMARY:
  definitely lost: 40 bytes in 1 blocks
```

Bryant and O'Hallaron, Computer Systems: A Programmer's Perspective, Third Edition

56

56